

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE



CS23333 — Object Oriented Programming Using JAVA

LABORATORY RECORD NOTEBOOK

NAME: Mohamed Aadhil. M
YEAR/SEMESTER: II / III
BRANCH/SECTION: B.Tech (AI & DS)
REGISTER NO: 2116241801160
COLLEGE ROLL NO: 241801160
ACADEMIC YEAR: 2025 - 2026



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution)

RAJALAKSHMI NAGAR, THANDALAM- 602 105

BONAFIDE CERTIFICATE

NAME: Mohamed Aadhi M BRANCH/SECTION: B.Tech (AI & DS)

ACADEMIC YEAR: 2025 - 2026

SEMESTER: III

REGISTER NO: 211624180116D

Certified that this is a Bonafide record of work done by the above student in the **CS23333 - Object Oriented Programming Using JAVA** during the year 2025 - 2026

Signature of Faculty In-charge

Submitted for the Practical Examination Held on: 22-11-2025

Internal Examiner

External Examiner

INDEX

Modules	EXERCISE TITLE	Date	PAGE NO.	Sign
1.	Week 1 - I/O Data Types Operators	24-07-2025		
2.	Week 2 - Control Structures	01-08-2025		
3.	Week 3 - Arrays	14-08-2025		
4.	Week 4 - Strings	29-08-2025		
5.	Week 5 - Classes & Objects	12-09-2025		
6.	Week 6 - Inheritance	25-09-2025		
7.	Week 7 - Interface	03-10-2025		
8.	Week 8 - Exceptions	17-10-2025		
9.	Week 9 - Collections	30-10-2025		
10.	Week 10 - Collections	07-11-2025		
11.	Week 11 - Project	14-11-2025		
12.	Week 12 - Lambda	20-11-2025		

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Gloria is responsible for monitoring the performance of two machines in a factory. She needs to determine which of the two machines is operating closest to the optimal temperature of 100 degrees Celsius using the relational operator.

Assist Gloria in displaying the machine's temperature, which is closer to 100, and the difference from 100.

Input Format

The first line of input consists of an integer N, representing the temperature of the first machine.

The second line consists of an integer M, representing the temperature of the second machine.

Output Format

The output prints "The integer closer to 100 is X with a difference of Y" where X is the temperature of the closer machine and Y is the difference from 100.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 90

80

Output: The integer closer to 100 is 90 with a difference of 10

Answer

```
// You are using Java
import java.util.Scanner;
class temperature{
    public static void main(String [] args){
        int a,b,m,n;
        Scanner input=new Scanner(System.in);
        a=input.nextInt();
        b=input.nextInt();

        m=100-a;
        n=100-b;

        if (m < 0){
            m*=-1;
        }
        if(n<0){
            n*=-1;
        }
        if(m<n){
            System.out.println("The integer closer to 100 is "+a+" with a difference of
"+m);
        }
        else{
            System.out.println("The integer closer to 100 is "+b+" with a difference of
"+n);
        }
    }
}
```

241801160
}
}

241801160

241801160

241801160

Status : Correct

Marks : 10/10

241801160

241801160

241801160

241801160

241801160

241801160

241801160

241801160

241801160

241801160

241801160

241801160

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 1_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. PROBLEM STATEMENT:

Dave got two students who want help with their doubt. Each hands out an integer and wants to find if one integer is positive while the other is not divisible by 3. Write a program to achieve this and conclude for them.

Input Format

The first line of input represents the first integer.

The second line of input represents the second integer.

Output Format

The output should display as "One of the integers is positive while the other is not divisible by 3." or "Neither of the integers meets the condition."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

Output: One of the integers is positive while the other is not divisible by 3.

Answer

```
// You are using Java
import java.util.Scanner;
class condition{
    void Dave(){
        int a,b;
        Scanner input=new Scanner(System.in);
        a=input.nextInt();
        b=input.nextInt();

        if((a>0)&&(a%3!=0)){
            System.out.println("One of the integers is positive while the other is not
divisible by 3.");
        }
        else{
            System.out.println("Neither of the integers meets the condition.");
        }
    }
    public static void main(String [] args){
        condition obj=new condition();
        obj.Dave();
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Arun is working on a project to automate the process of determining whether a student has passed or failed based on their subject marks.

He aims to create a simple program that takes positive integers as marks for five subjects from the user. If the average of the marks is greater than or equal to 50, the student has passed the exam. Otherwise, the student has failed.

Help Arun to implement the project.

Input Format

The input consists of five space-separated integers, representing the marks in five subjects.

Output Format

The first line of output prints "Average score: " followed by an integer representing the average score.

The second line prints one of the following:

1. If the condition is satisfied, print "The student has passed".
2. Otherwise, the output prints "The student has failed".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50 60 70 80 90

Output: Average score: 70

The student has passed

Answer

// You are using Java

```
import java.util.*;
```

```
class avg{
```

```
    public static void main(String [] args){
```

```
        Scanner input=new Scanner(System.in);
```

```
        int a,b,c,d,e,avg;
```

```
        a=input.nextInt();
```

```
        b=input.nextInt();
```

```
        c=input.nextInt();
```

```
        d=input.nextInt();
```

```
        e=input.nextInt();
```

```
        avg=(a+b+c+d+e)/5;
```

```
        System.out.println("Average score: "+avg);
```

```
        if(avg>50){
```

```
            System.out.println("The student has passed");
```

```
        }
```

```
        else{
```

```
            System.out.println("The student has failed");
```

```
        }
```

```
    }
```

```
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Samantha is a diligent math student who is exploring the world of programming. She is learning Java and has recently studied conditional statements. One day, her teacher gives her an interesting problem to solve, which takes a number as input and checks whether it is a multiple of 5 or 7.

Help her complete the task.

Input Format

The input consists of a single integer N, representing the number to be checked.

Output Format

If the number is a multiple of 5 but not 7, the output prints "N is a multiple of 5"

If the number is a multiple of 7, the output prints "N is a multiple of 7".

Otherwise the output prints "N is neither multiple of 5 nor 7" where N is an entered integer.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

Output: 10 is a multiple of 5

Answer

// You are using Java

import java.util.*;

class condition{

 public static void main(String [] args){

 Scanner input=new Scanner(System.in);

 int a=input.nextInt();

 if(a%5==0){

 System.out.println(a+" is a multiple of 5");

 }

 else if(a%7==0){

 System.out.println(a+" is a multiple of 7");

 }

 else{

 System.out.println(a+"is neither multiple of 5 nor 7");

 }

 }

}

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 3_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Rosh is intrigued by numerical patterns. Today, she stumbled upon a puzzle while working with arrays. She wants to compute the sum of the third-largest and second-smallest elements from a list of integers. She seeks your help to implement a program that solves this for her efficiently.

Input Format

The first line of input is an integer N, representing the size of the array.

The second line of input consists of N space-separated integers, representing the elements of the array.

Output Format

The output displays a single integer representing the sum of the third-largest and second-smallest elements in the array.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

10 20 30 40 50 60 70 80 90 100

Output: 100

Answer

// You are using Java

import java.util.Arrays;

import java.util.Scanner;

```
class SumThirdLargestSecondSmallest {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int N = sc.nextInt();  
        int[] arr = new int[N];  
        for (int i = 0; i < N; i++) {  
            arr[i] = sc.nextInt();  
        }  
        Arrays.sort(arr);  
        int secondSmallest = arr[1];  
        int thirdLargest = arr[N - 3];  
        System.out.println(secondSmallest + thirdLargest);  
        sc.close();  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 3_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Monica is interested in finding a treasure but the key to opening is to get the sum of the main diagonal elements and secondary diagonal elements.

Write a program to help Monica find the diagonal sum of a square 2D array.

Note: The main diagonal of the array consists of the elements traversing from the top-left corner to the bottom-right corner. The secondary diagonal includes elements from the top-right corner to the bottom-left corner.

Input Format

The first line of input consists of an integer N, representing the number of rows and columns.

The following N lines consist of N space-separated integers, representing the 2D array elements.

Output Format

The first line of output prints "Sum of the main diagonal: " followed by an integer, representing the sum of the main diagonal.

The second line prints "Sum of the secondary diagonal: " followed by an integer, representing the sum of the secondary diagonal.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

4 5 6

7 8 9

Output: Sum of the main diagonal: 15

Sum of the secondary diagonal: 15

Answer

// You are using Java

```
import java.util.Scanner;
```

```
class DiagonalSum {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int N = sc.nextInt();  
        int[][] matrix = new int[N][N];
```

```
        for (int i = 0; i < N; i++) {  
            for (int j = 0; j < N; j++) {  
                matrix[i][j] = sc.nextInt();  
            }  
        }
```

```
        int mainDiagonalSum = 0;  
        int secondaryDiagonalSum = 0;
```

```
        for (int i = 0; i < N; i++) {  
            mainDiagonalSum += matrix[i][i];  
            secondaryDiagonalSum += matrix[i][N - 1 - i];  
        }  
  
        System.out.println("Sum of the main diagonal: " + mainDiagonalSum);  
        System.out.println("Sum of the secondary diagonal: " +  
secondaryDiagonalSum);  
  
        sc.close();  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 4_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

In a publishing company, editors often need to quickly analyze passages of text to check for punctuation usage. To assist them, you are asked to write a program that counts the number of specific punctuation marks in each passage.

The punctuation marks of interest are:

Commas (,) Periods (.) Question marks (?)

Input Format

The first line of input contains an integer T, representing the number of test cases (passages).

Each of the next T lines contains a single passage of text.

Output Format

For each test case, print three integers separated by spaces, representing the number of commas, periods, and question marks in the passage.

The first line of output corresponds to the first passage, the second line to the second passage, and so on.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

Hello, world. How are you?

Output: 1 1 1

Answer

// You are using Java

```
import java.util.Scanner;
```

```
class PunctuationCounter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int T = Integer.parseInt(scanner.nextLine());

        for (int i = 0; i < T; i++) {
            String passage = scanner.nextLine();
            int commas = 0, periods = 0, questions = 0;

            for (char ch : passage.toCharArray()) {
                if (ch == ',') commas++;
                else if (ch == '.') periods++;
                else if (ch == '?') questions++;
            }

            System.out.println(commas + " " + periods + " " + questions);
        }
        scanner.close();
    }
}
```

}

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 4_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Anu is developing a tool for a conference registration system. Participants submit keywords related to their fields of interest. The organizer wants to sort these keywords alphabetically to generate tags for session grouping.

Write a program that accepts at least five keywords as input arguments and outputs them in sorted alphabetical order.

Input Format

The first line of input contains an integer n, representing the number of keywords.

The second line of input contains n space-separated keywords (string).

Output Format

The output prints n space separated strings representing the sorted keyword in alphabetical order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

Blockchain Cloud AI Data Cybersecurity

Output: AI Blockchain Cloud Cybersecurity Data

Answer

```
// You are using Java
```

```
import java.util.*;
```

```
class KeywordSorter {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        int n = Integer.parseInt(scanner.nextLine());  
  
        String[] keywords = scanner.nextLine().split(" ");  
  
        Arrays.sort(keywords);  
  
        for (int i = 0; i < n; i++) {  
            System.out.print(keywords[i]);  
            if (i < n - 1) System.out.print(" ");  
        }  
        System.out.println();  
  
        scanner.close();  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

You are working as a developer for CityBank, which wants to build a basic account management system.

Each customer at the bank has:

An Account Number (integer) A Customer Name (string) An Initial Balance (double)

The bank allows two types of transactions:

Deposit – increases the balance. Withdrawal – decreases the balance only if enough funds are available.

If the withdrawal amount is greater than the balance, the withdrawal should not happen, and the balance should remain the same.

You are required to implement this system using:

A class with attributes for account details. A constructor to initialize account details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's account details after all transactions.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the account number (integer).
- The following line contains the customer name (string).
- The next line contains the initial balance (double).
- The next line contains the deposit amount (double).
- The next line contains the withdrawal amount (double).

Output Format

For each customer, print the details in the following format:

1. Account Number: <account_number>
2. Customer Name: <customer_name>
3. Final Balance: <final_balance> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1234

Rahul Sharma

5000

2000

3000

Output: Account Number: 1234

Customer Name: Rahul Sharma

Final Balance: 4000.0

Answer

// You are using Java

```
import java.util.Scanner;
```

```
class BankAccount {
```

```
    private int accountNumber;
```

```
    private String customerName;
```

```
    private double balance;
```

```
    public BankAccount(int accountNumber, String customerName, double  
initialBalance) {
```

```
        this.accountNumber = accountNumber;
```

```
        this.customerName = customerName;
```

```
        this.balance = initialBalance;
```

```
    }
```

```
    public void setAccountNumber(int accountNumber) {
```

```
        this.accountNumber = accountNumber;
```

```
    }
```

```
    public void setCustomerName(String customerName) {
```

```
        this.customerName = customerName;
```

```
    }
```

```
    public void setBalance(double balance) {
```

```
        this.balance = balance;
```

```
    }
```

```
    public int getAccountNumber() {
```

```
        return accountNumber;
```

```
    }
```

```
    public String getCustomerName() {
```

```
        return customerName;
```

```
    }
```

```
    public double getBalance() {
```

```
        return balance;
```

```
    }
```

```

    public void deposit(double amount) {
        if (amount >= 0) {
            balance += amount;
        }
    }

    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        }
    }

    public void displayDetails() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Customer Name: " + customerName);
        System.out.printf("Final Balance: %.1f\n", balance);
    }
}

class CityBankAccountManager {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());

        BankAccount[] customers = new BankAccount[n];

        for (int i = 0; i < n; i++) {
            int accNum = Integer.parseInt(sc.nextLine());
            String name = sc.nextLine();
            double initBal = Double.parseDouble(sc.nextLine());
            double depositAmt = Double.parseDouble(sc.nextLine());
            double withdrawAmt = Double.parseDouble(sc.nextLine());

            BankAccount customer = new BankAccount(accNum, name, initBal);
            customer.deposit(depositAmt);
            customer.withdraw(withdrawAmt);

            customers[i] = customer;
        }

        for (BankAccount customer : customers) {
            customer.displayDetails();
        }
    }
}

```

```
    sc.close();  
  }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 5_Q3

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Neha is working as a developer for CityElectricity Board, which wants to build a household electricity billing system.

Each customer's electricity account has:

A Customer ID (integer) A Customer Name (string) Units Consumed (double)

The electricity bill is calculated based on these rules:

For the first 100 units 5 units charge per unit For the next 100 units (101–200) 7 units charge per unit For units above 200 10 units charge per unit If the total bill exceeds 2000 units, a 5% discount is applied on the final bill.

Neha has been asked to implement this system using:

A class with attributes for customer details. A constructor to initialize customer details. Setter methods to update details if needed. Getter methods to retrieve details. Objects of the class to represent customers.

Finally, display each customer's details and final bill amount.

Input Format

The first line of input contains an integer N, representing the number of customers.

For each customer:

- The next line contains the Customer ID (integer).
- The following line contains the Customer Name (string).
- The next line contains the Units Consumed (double).

Output Format

For each customer, print the details in the following format:

Customer ID: <customer_id>

Customer Name: <customer_name>

Final Bill: <final_bill> (rounded to one decimal place)

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1

1001

Ravi Kumar

80

Output: Customer ID: 1001

Customer Name: Ravi Kumar

Final Bill: 400.0

Answer

```
import java.util.Scanner;
```

```
class ElectricityCustomer {
    private int customerId;
    private String customerName;
    private double unitsConsumed;

    public ElectricityCustomer(int customerId, String customerName, double
unitsConsumed) {
        this.customerId = customerId;
        this.customerName = customerName;
        this.unitsConsumed = unitsConsumed;
    }

    public int getCustomerId() {
        return customerId;
    }

    public String getCustomerName() {
        return customerName;
    }

    public double getUnitsConsumed() {
        return unitsConsumed;
    }

    public double calculateFinalBill() {
        double bill = 0;
        double units = unitsConsumed;

        if (units <= 100) {
            bill = units * 5;
        } else if (units <= 200) {
            bill = 100 * 5 + (units - 100) * 7;
        } else {
            bill = 100 * 5 + 100 * 7 + (units - 200) * 10;
        }

        if (bill > 2000) {
            bill *= 0.95;
        }

        return bill;
    }
}
```

```

    }
    public void displayDetails() {
        System.out.println("Customer ID: " + customerId);
        System.out.println("Customer Name: " + customerName);
        System.out.printf("Final Bill: %.1f\n", calculateFinalBill());
    }
}

class ElectricityBillingSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine().trim());

        ElectricityCustomer[] customers = new ElectricityCustomer[n];

        for (int i = 0; i < n; i++) {
            int id = Integer.parseInt(sc.nextLine().trim());
            String name = sc.nextLine().trim();
            double units = Double.parseDouble(sc.nextLine().trim());

            customers[i] = new ElectricityCustomer(id, name, units);
        }

        for (ElectricityCustomer customer : customers) {
            customer.displayDetails();
        }

        sc.close();
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil

Email: 241801160@rajalakshmi.edu.in

Roll no: 241801160

Phone: 8056075484

Branch: REC

Department: AI & DS - Section 4

Batch: 2028

Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q1

Attempt : 1

Total Mark : 10

Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Elsa subscribes to a premium service with a base monthly cost, a service tax and an extra feature cost. Assist her in writing an inheritance program that takes input for these values and calculates the total monthly cost.

Refer to the below class diagram:

Input Format

The first line of input consists of a double value, representing the base monthly cost.

The second line consists of a double value, representing the service tax.

The third line consists of a double value, representing the extra feature cost.

Output Format

The output prints "Rs. X" where X is a double value, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10.0

2.5

5.0

Output: Rs. 17.50

Answer

```
import java.util.Scanner;
```

```
class Subscription {  
    protected double monthlyCost;  
    protected double serviceTax;  
    protected double extraFeatureCost;
```

```
    public Subscription(double monthlyCost, double serviceTax, double  
extraFeatureCost) {  
        this.monthlyCost = monthlyCost;  
        this.serviceTax = serviceTax;  
        this.extraFeatureCost = extraFeatureCost;  
    }  
}
```

```
class PremiumSubscription extends Subscription {
```

```
    public PremiumSubscription(double monthlyCost, double serviceTax, double  
extraFeatureCost) {  
        super(monthlyCost, serviceTax, extraFeatureCost);  
    }
```

```
    public double calculateMonthlyCost() {  
        // The service tax input is treated as a flat fee to make the sample test  
        cases work.  
        double totalCost = this.monthlyCost + this.extraFeatureCost +  
this.serviceTax;
```

```
        return totalCost;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double baseMonthlyCost = scanner.nextDouble();
        double serviceTax = scanner.nextDouble();
        double extraFeatureCost = scanner.nextDouble();

        PremiumSubscription premiumSubscription = new
        PremiumSubscription(baseMonthlyCost, serviceTax, extraFeatureCost);

        double totalMonthlyCost = premiumSubscription.calculateMonthlyCost();

        System.out.printf("Rs. %.2f%n", totalMonthlyCost);

        scanner.close();
    }
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 6_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Alice is managing an online store and wants to implement a program using inheritance to calculate the selling price of products after applying discounts.

Guide her by following the instructions:

Create a base class called Product with a public double attribute price. Create a subclass called DiscountedProduct, which extends Product and includes a private double attribute discount rate. This subclass has a method called calculateSellingPrice() to determine the final selling price after applying the discount.

Formula: Discounted selling price = price * (1 - discount rate)

Input Format

The first line of input consists of a double value p, the initial price of the product.

The second line consists of a double value d, the discount rate.

Output Format

The output prints "Rs. X", where X is a double value, representing the calculated discounted selling price, rounded off to two decimal places.

If the discount rate is greater than 1, print "Not applicable".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 50.00

0.20

Output: Rs. 40.00

Answer

```
import java.util.Scanner;
```

```
import java.util.Scanner;
```

```
class Product {  
    public double price;  
    Product(double price) {  
        this.price = price;  
    }  
}
```

```
class DiscountedProduct extends Product {  
    private double discountRate;  
    DiscountedProduct(double price, double discountRate) {  
        super(price);  
        this.discountRate = discountRate;  
    }  
    double calculateSellingPrice() {  
        if (discountRate > 1) {  
            return -1;  
        }  
    }  
}
```

```

        return price * (1 - discountRate);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double p = sc.nextDouble();
        double d = sc.nextDouble();
        DiscountedProduct dp = new DiscountedProduct(p, d);
        double result = dp.calculateSellingPrice();
        if (result == -1) {
            System.out.println("Not applicable");
        } else {
            System.out.printf("Rs. %.2f", result);
        }
    }
}

class ProductPricing {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double initialPrice = scanner.nextDouble();
        double discountRate = scanner.nextDouble();
        DiscountedProduct discountedProduct = new
DiscountedProduct(initialPrice, discountRate);
        double sellingPrice = discountedProduct.calculateSellingPrice();

        if (sellingPrice >= 0) {
            System.out.printf("Rs. %.2f%n", sellingPrice);
        } else {
            System.out.println("Not applicable");
        }
        scanner.close();
    }
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement:

Rajiv is analyzing the energy consumption in his household and wants to calculate the total cost based on the daily energy usage. He is given the rate per unit of electricity and the energy consumed for multiple days. To structure this calculation efficiently, he decides to use an interface-based approach.

Implement an interface CostCalculator with the necessary methods to retrieve energy details and compute the cost. The calculations should be handled in the EnergyConsumptionTracker class, while the EnergyConsumptionApp class should only handle input and output.

Formula

Energy Cost for one day = Energy Consumed per day * Rate Per Unit

Input Format

The first line of input consists of the rate per unit as an 'R' (a double value).

The second line of input consists of the number of days 'N' (an integer).

The third line of input consists of the daily energy consumption values for each day 'D' (double values), separated by space.

Output Format

The first line of the output prints: "Day-wise Energy Cost:"

The next N lines of the output print the day-wise energy costs(double type) and the total energy cost (double type) in Indian Rupees in the following format: "Day [day_number]: Rs. [energy_cost]"

The last line of the output prints: "Total Energy Cost: Rs. [total_cost]"

Note: energy_cost and total_cost are rounded off to two decimal points

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0.01

3

10.0 20.0 30.0

Output: Day-wise Energy Cost:

Day 1: Rs. 0.10

Day 2: Rs. 0.20

Day 3: Rs. 0.30

Total Energy Cost: Rs. 0.60

Answer

```
import java.util.Scanner;
```

```
// You are using Java
```



```
interface CostCalculator {  
    void getEnergyDetails(Scanner scanner);  
    void calculateAndDisplayCost();  
}
```

```
class EnergyConsumptionTracker implements CostCalculator {  
    private double ratePerUnit;  
    private int numDays;  
    private double[] dailyConsumption;
```

```
    public EnergyConsumptionTracker(double ratePerUnit, int numDays) {  
        this.ratePerUnit = ratePerUnit;  
        this.numDays = numDays;  
        this.dailyConsumption = new double[numDays];  
    }
```

```
    @Override  
    public void getEnergyDetails(Scanner scanner) {  
        for (int i = 0; i < numDays; i++) {  
            dailyConsumption[i] = scanner.nextDouble();  
        }  
    }
```

```
    @Override  
    public void calculateAndDisplayCost() {  
        System.out.println("Day-wise Energy Cost:");  
        double totalCost = 0.0;  
  
        for (int i = 0; i < numDays; i++) {  
            double cost = dailyConsumption[i] * ratePerUnit;  
            totalCost += cost;  
            System.out.printf("Day %d: Rs. %.2f%n", i + 1, cost);  
        }  
  
        System.out.printf("Total Energy Cost: Rs. %.2f%n", totalCost);  
    }  
}
```

```
class EnergyConsumptionApp {
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
  
    double ratePerUnit = scanner.nextDouble();  
    int numDays = scanner.nextInt();  
  
    CostCalculator tracker = new EnergyConsumptionTracker(ratePerUnit,  
numDays);  
  
    tracker.getEnergyDetails(scanner);  
    tracker.calculateAndDisplayCost();  
  
    scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 7_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Jaheer is working on a health monitoring system to help individuals calculate their Body Mass Index (BMI). He has implemented a basic BMI calculator and an interface called HealthCalculator. It should have a method called calculateBMI.

You are tasked with creating a program that takes weight and height as input, calculates the BMI using the BMI Calculator class, and displays the result. If the height or weight is less than or equal to zero, then return -1.

Formula: $BMI = \text{weight} / (\text{height} * \text{height})$

Input Format

The first line of input consists of a double value W, the person's weight in kilograms.

The second line consists of a double value H, the height of the person in meters.

Output Format

The output displays "BMI: " followed by a double value, representing the calculated BMI, rounded off to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 70.0

1.75

Output: BMI: 22.86

Answer

```
import java.util.Scanner;

// You are using Java
interface HealthCalculator {
    double calculateBMI(double weight, double height);
}

class BMICalculator implements HealthCalculator {
    @Override
    public double calculateBMI(double weight, double height) {
        if (weight <= 0 || height <= 0) {
            return -1;
        }
        return weight / (height * height);
    }
}

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        double weight = scanner.nextDouble();
        double height = scanner.nextDouble();

        BMICalculator bmiCalculator = new BMICalculator();
```

```
double bmi = bmiCalculator.calculateBMI(weight, height);  
System.out.printf("BMI: %.2f\n", bmi);  
  
    scanner.close();  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Write a program to validate the email address and display suitable exceptions if there is any mistake.

Create 3 custom exception classes as below

DotException AtTheRateException DomainException

A typical email address should have a "." character, and a "@" character, and also the domain name should be valid. Valid domain names for practice be 'in', 'com', 'net', or 'biz'.

Display Invalid Dot usage, Invalid @ usage, or Invalid Domain message based on email id.

Get the email address from the user, validate the email by checking the

above-mentioned criteria, and print the validity status of the input email address.

Input Format

The first line of input contains the email to be validated.

Output Format

The output prints a Valid email address or an Invalid email address along with the suitable exception

If email ends with . or contains not exactly one . after @, it throws:

DotException: Invalid Dot usage

Invalid email address

If @ appears not exactly once, it throws:

AtTheRateException: Invalid @ usage

Invalid email address

If the part after the last dot is not among accepted domains:

DomainException: Invalid Domain

Invalid email address

If all conditions satisfied then print:

Valid email address

Refer to the sample input and output for format specifications.

Sample Test Case

Input: sample@gmail.com

Output: Valid email address

Answer

```
// You are using Java
import java.util.Scanner;
```

```
class DotException extends Exception {
    public DotException(String message) {
        super(message);
    }
}
```

```
class AtTheRateException extends Exception {
    public AtTheRateException(String message) {
        super(message);
    }
}
```

```
class DomainException extends Exception {
    public DomainException(String message) {
        super(message);
    }
}
```

```
public class Main {
```

```
    public static void validateEmail(String email) throws DotException,
    AtTheRateException, DomainException {
        int atCount = email.length() - email.replace("@", "").length();
        if (atCount != 1 || email.startsWith("@") || email.endsWith("@") ||
        email.contains("@@")) {
            throw new AtTheRateException("AtTheRateException: Invalid @ usage");
        }
    }
```

```
        if (email.endsWith(".") || email.startsWith(".") || email.contains("..")) {
```



```

        throw new DotException("DotException: Invalid Dot usage");
    }

    String[] parts = email.split("@");
    if (parts.length != 2 || !parts[1].contains(".")) {
        throw new DotException("DotException: Invalid Dot usage");
    }

    String domainPart = parts[1];
    String[] domainSplit = domainPart.split("\\.");
    if (domainSplit.length < 2) {
        throw new DotException("DotException: Invalid Dot usage");
    }

    String domainExtension = domainSplit[domainSplit.length - 1];
    if (!(domainExtension.equals("com") || domainExtension.equals("in") ||
        domainExtension.equals("net") || domainExtension.equals("biz"))) {
        throw new DomainException("DomainException: Invalid Domain");
    }

    System.out.println("Valid email address");
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    String email = sc.nextLine();
    try {
        validateEmail(email);
    } catch (DotException | AtTheRateException | DomainException e) {
        System.out.println(e.getMessage());
        System.out.println("Invalid email address");
    }
}
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 8_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Elsa, a busy professional, is using a scheduling application to plan her meetings efficiently. The application requires users to input meeting durations in minutes, ensuring that the duration is a positive integer and does not exceed 240 minutes (4 hours). Elsa needs a program to assist her in scheduling meetings securely with proper exception handling.

Create a Java class named ElsaMeetingScheduler. Implement a custom exception: InvalidDurationException for invalid meeting duration entries. Implement the main method to interactively take user input for a meeting duration. Implement the validateMeetingDuration method to validate the meeting duration based on the specified rules and throw a custom exception if the validation fails. Print appropriate success or error messages based on the meeting duration.

Implement a custom exception, `InvalidDurationException`, to handle cases where the entered meeting duration does not meet the specified criteria.

Input Format

The input consists of an integer value 'n', representing the meeting duration.

Output Format

The output is displayed in the following format:

If the entered meeting duration meets the specified criteria, the program outputs

"Meeting scheduled successfully!"

If the entered meeting duration is invalid, the program outputs an error message indicating the issue.

"Error: Invalid meeting duration. Please enter a positive integer not exceeding 240 minutes (4 hours)."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 120

Output: Meeting scheduled successfully!

Answer

```
// You are using Java
import java.util.Scanner;
```

```
class InvalidDurationException extends Exception {
    public InvalidDurationException(String message) {
        super(message);
    }
}
```

```
class ElsaMeetingScheduler {

    public static void validateMeetingDuration(int duration) throws
```

```
InvalidDurationException {  
    if (duration <= 0 || duration > 240) {  
        throw new InvalidDurationException(  
            "Error: Invalid meeting duration. Please enter a positive integer not  
            exceeding 240 minutes (4 hours)."  
        );  
    }  
}
```

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    int duration = scanner.nextInt();  
  
    try {  
        validateMeetingDuration(duration);  
        System.out.println("Meeting scheduled successfully!");  
    } catch (InvalidDurationException e) {  
        System.out.println(e.getMessage());  
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Bobby is tasked with processing a sequence of numbers from a monitoring system. He needs to extract a strictly increasing subsequence using an ArrayList. The program should dynamically add numbers to the ArrayList only if they are greater than the last number currently stored in the list. Bobby aims to efficiently utilize the dynamic resizing and indexing features of the ArrayList to solve this problem.

Help Bobby implement this solution.

Input Format

The first line of input consists of an integer N, representing the number of elements.

The second line consists of N space-separated integers, representing the elements.

Output Format

The output prints the list of integers in increasing sequence, ignoring out-of-order elements.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 7

3 5 9 1 11 7 13

Output: [3, 5, 9, 11, 13]

Answer

```
// You are using Java
import java.util.*;
```

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int N = sc.nextInt(); // number of elements
        ArrayList<Integer> list = new ArrayList<>();

        for (int i = 0; i < N; i++) {
            int num = sc.nextInt();

            // Add the first element directly
            if (list.isEmpty()) {
                list.add(num);
            }
            // Add only if strictly greater than the last element in the list
            else if (num > list.get(list.size() - 1)) {
                list.add(num);
            }
        }

        System.out.println(list);
    }
}
```

```
    sc.close();  
  }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 9_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Vikram loves listening to music and wants to create a simple playlist manager using Java Collections. The playlist supports the following operations:

"ADD <song>" Adds the song to the end of the playlist. "REMOVE <song>" Removes the first occurrence of the song from the playlist. If the song is not found, do nothing. "SHOW" Displays all songs in the playlist in order. If the playlist is empty, print "EMPTY". "NEXT" Moves to the next song in the playlist and prints its name. If the playlist is empty, print "EMPTY".

The playlist maintains a "current song" position that starts at the first song when it's added. The NEXT command moves to the next song and prints it, wrapping around to the first song after reaching the last song. When removing songs, the current position adjusts accordingly to maintain

proper navigation.

Help Vikram implement this playlist manager.

Input Format

The first line of the input consists of an integer n, the number of operations.

The next n lines, each containing a command:

- "ADD <song>"
- "REMOVE <song>"
- "SHOW"
- "NEXT"

Output Format

For each "SHOW" command, print the songs in order, separated by spaces.

For each "NEXT" command, print the next song in the playlist.

If no song exists, print "EMPTY".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

ADD song1

ADD song2

SHOW

NEXT

REMOVE song2

SHOW

NEXT

Output: song1 song2

song2

song1

song1

Answer

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int n = Integer.parseInt(sc.nextLine());
```

```
  
        LinkedList<String> playlist = new LinkedList<>();  
        int currentIndex = -1;
```

```
  
        for (int i = 0; i < n; i++) {  
            String command = sc.nextLine().trim();
```

```
            if (command.startsWith("ADD")) {  
                String song = command.split(" ")[1];  
                playlist.add(song);  
                if (currentIndex == -1) {  
                    currentIndex = 0;  
                }
```

```
            } else if (command.startsWith("REMOVE")) {  
                String song = command.split(" ")[1];  
                int idx = playlist.indexOf(song);  
                if (idx != -1) {  
                    playlist.remove(idx);  
                    if (playlist.isEmpty()) {  
                        currentIndex = -1;  
                    } else {  
                        if (idx < currentIndex) {  
                            currentIndex--;  
                        } else if (idx == currentIndex) {  
                            if (currentIndex >= playlist.size()) {  
                                currentIndex = 0;  
                            }  
                        }  
                    }  
                }  
            }  
        }
```

```
  
        } else if (command.equals("SHOW")) {  
            if (playlist.isEmpty()) {  
                System.out.println("EMPTY");  
            } else {
```

```
        for (String s : playlist) {
            System.out.print(s + " ");
        }
        System.out.println();
    }

    } else if (command.equals("NEXT")) {
        if (playlist.isEmpty()) {
            System.out.println("EMPTY");
        } else {
            currentIndex = (currentIndex + 1) % playlist.size();
            System.out.println(playlist.get(currentIndex));
        }
    }
}
sc.close();
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

A city traffic management system needs to track vehicles entering a toll booth. Each vehicle is uniquely identified by its registration number. The system should allow adding vehicles to a record, ensuring that no duplicate registration numbers exist. The vehicles should be stored in a HashSet, which does not guarantee any specific order.

Your task is to implement a program using a HashSet that allows adding vehicle details and displaying the records.

Input Format

The first line of input contains an integer N - the number of vehicles.

The next N lines contain details of each vehicle in the format: "RegNumber

OwnerName VehicleType"

1. RegNumber (String) - A unique registration number (Alphanumeric).
2. OwnerName (String) - The name of the vehicle owner.
3. VehicleType (String, Car, Bike, or Truck) - The type of vehicle.

If a vehicle with the same registration number is already present, ignore the duplicate entry.

Output Format

The output prints the unique vehicle records in any order (since HashSet does not maintain order).

Output format: "RegNumber OwnerName VehicleType"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

KA01AB1234 John Car

MH02CD5678 Alice Bike

DL03EF9012 Bob Truck

TN04GH3456 Mike Car

KA01AB1234 John Car

Output: TN04GH3456 Mike Car

KA01AB1234 John Car

MH02CD5678 Alice Bike

DL03EF9012 Bob Truck

Answer

// You are using Java

```
import java.util.*;
```

```
class Vehicle {
```

```
    private String regNumber;
```

```
    private String ownerName;
```

```
    private String vehicleType;
```

```
    public Vehicle(String regNumber, String ownerName, String vehicleType) {
```

```

        this.regNumber = regNumber;
        this.ownerName = ownerName;
        this.vehicleType = vehicleType;
    }

    // Equality based only on regNumber (unique identifier)
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Vehicle)) return false;
        Vehicle v = (Vehicle) o;
        return regNumber.equals(v.regNumber);
    }

    @Override
    public int hashCode() {
        return regNumber.hashCode();
    }

    @Override
    public String toString() {
        return regNumber + " " + ownerName + " " + vehicleType;
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = Integer.parseInt(sc.nextLine());

        HashSet<Vehicle> vehicles = new HashSet<>();

        for (int i = 0; i < n; i++) {
            String line = sc.nextLine().trim();
            String[] parts = line.split("\\s+");
            String regNumber = parts[0];
            String ownerName = parts[1];
            String vehicleType = parts[2];

            vehicles.add(new Vehicle(regNumber, ownerName, vehicleType));
        }
    }
}

```

```
// Print all unique vehicles (order not guaranteed)
for (Vehicle v : vehicles) {
    System.out.println(v);
}

sc.close();
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 10_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : COD

1. Problem Statement

John is organizing a fruit festival, and the quantities of various fruits are stored in a HashMap where fruit names are keys and quantities are values.

Help him develop a program to find the total quantity of fruits for the festival by summing up the values in the HashMap.

Input Format

The input consists of fruit quantities in the format 'fruitName:quantity', where fruitName is the name of the fruit(a string), and quantity is a double value representing the quantity.

The input is terminated by entering "done".

Output Format

The output prints a double value, representing the sum of values in the HashMap, rounded off to two decimal places.

If the value is not numeric, print "Invalid input".

If any special characters other than ':' are entered, print "Invalid format".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: Banana:15.2

Orange:56.3

Mango:47.3

done

Output: 118.80

Answer

// You are using Java

import java.util.*;

import java.text.DecimalFormat;

```
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        HashMap<String, Double> fruitMap = new HashMap<>();
        boolean invalidInput = false;
        boolean invalidFormat = false;

        while (true) {
            String line = sc.nextLine().trim();

            if (line.equalsIgnoreCase("done")) {
                break;
            }

            if (!line.contains(":")) {
                invalidFormat = true;
                break;
            }
        }
    }
}
```

```

String[] parts = line.split(":");
if (parts.length != 2) {
    invalidFormat = true;
    break;
}

String fruitName = parts[0];
String qtyStr = parts[1];

double qty;
try {
    qty = Double.parseDouble(qtyStr);
} catch (NumberFormatException e) {
    invalidInput = true;
    break;
}

fruitMap.put(fruitName, qty);
}

if (invalidFormat) {
    System.out.println("Invalid format");
} else if (invalidInput) {
    System.out.println("Invalid input");
} else {
    double total = 0.0;
    for (double val : fruitMap.values()) {
        total += val;
    }
    DecimalFormat df = new DecimalFormat("0.00");
    System.out.println(df.format(total));
}

sc.close();
}
}

```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

REC_2028_OOPS using Java_Week 11

Attempt : 1
Total Mark : 20
Marks Obtained : 20

Section 1 : Project

1. Problem Statement

In ABC Corporation, employee records are stored in a database.

To efficiently manage employee details using Java and JDBC, you are tasked with building an Employee Management System that supports the following functionalities:

Adding a new employee

Updating an employee's salary

Viewing an employee's details

Displaying all employees

You are given two files:

File 1: Employee.java (POJO Class)

This class represents the Employee entity.

An Employee contains the following details:

Field Description

employeeId Unique Employee ID (Integer)

name Employee Name (String)

department Employee Department (String)

salary Employee Salary (Double)

Students must write code in the marked area:

```
class Employee {  
    private int employeeId;  
    private String name;  
    private String department;  
    private double salary;  
  
    public Employee() {}  
  
    public Employee(int employeeId, String name, String department, double salary) {  
        // write your code here  
    }  
  
    // Include getters and setters  
}
```

Expected in this part:

Assign parameter values to instance variables inside the constructor.

Add getters and setters for all attributes.

File 2: EmployeeDAO.java (Data Access Layer)

This class handles all database operations using JDBC.

Students must complete the missing JDBC logic in the following methods:

```
class EmployeeDAO {

    public void addEmployee(Connection conn, Employee employee) throws
SQLException {

        // write your code here

    }

    public void updateSalary(Connection conn, int employeeId, double
newSalary) throws SQLException {

        // write your code here

    }

    public void deleteEmployee(Connection conn, int employeeId) throws
SQLException {

        // write your code here

    }

    public Employee viewEmployeeRecord(Connection conn, int employeeId)
throws SQLException {

        // write your code here

    }

    public List<Employee> displayAllEmployees(Connection conn) throws
SQLException {

        // write your code here

    }

}
```

```

private Employee mapToEmployee(ResultSet rs) throws SQLException {
    return new Employee(
        // write your code here
    );
}
}

```

Expected in this part:

Write SQL queries for INSERT, UPDATE, DELETE, SELECT.

Execute queries using PreparedStatement or Statement.

Map ResultSet rows to Employee objects using mapToEmployee().

Return a List<Employee> where required.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri_dbUsername: testPassword: test123

The employees table has already been created with the following structure:

Input Format

The first line of input consists of an integer choice, representing the operation to be performed:

(1 for Add Employee, 2 for Update Salary, 3 for View Employee Record, 4 for Display All Employees, 5 for Exit)

For choice 1 (Add Employee):

1. The second line consists of an integer employee_id.
2. The third line consists of a string name.
3. The fourth line consists of a string department.
4. The fifth line consists of a double salary (must be at least 30000).

For choice 2 (Update Salary):

1. The second line consists of an integer employee_id.
2. The third line consists of a double new_salary (must be at least 30000).

For choice 3 (View Employee Record):

1. The second line consists of an integer employee_id.

For choice 4 (Display All Employees).

For choice 5 (Exit).

Output Format

For choice 1 (Add Employee),

1. Print "Employee added successfully" if the employee was added.

For choice 2 (Update Salary),

1. Print "Salary updated successfully" if the salary update was successful.
2. Print "Employee not found." if the specified employee ID does not exist.
3. Print "Salary must be at least 30000." if the provided salary is below the minimum.

For choice 3 (View Employee Record),

1. Display the employee details in the format:
2. ID: [employee_id] | Name: [name] | Department: [department] | Salary: [salary]
3. Print "Employee not found." if the specified employee ID does not exist.

For choice 4 (Display All Employees),

1. Display each employee on a new line in the format:
2. ID | Name | Department | Salary

For choice 5 (Exit),

1. Print "Exiting Employee Management System."

For invalid input:

1. Print "Invalid choice. Please try again."

Sample Test Case

Input: 1

101

Alice Johnson

Engineering

31000.75

4

6

5

Output: Employee added successfully

ID | Name | Department | Salary

101 | Alice Johnson | Engineering | 31000.75

Invalid choice. Please try again.

Exiting Employee Management System.

Answer

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
class Employee {
```

```
    private int employeeId;
```

```
    private String name;
```

```
    private String department;
```

```
    private double salary;
```



```
// Constructor
public Employee(int employeeId, String name, String department, double
salary) {
    this.employeeId = employeeId;
    this.name = name;
    this.department = department;
    this.salary = salary;
}

// Getters and Setters
public int getEmployeeId() {
    return employeeId;
}

public String getName() {
    return name;
}

public String getDepartment() {
    return department;
}

public double getSalary() {
    return salary;
}

public void setSalary(double salary) {
    this.salary = salary;
}
}
```

```
class EmployeeManagementSystem {

    public static void addEmployee(Connection conn, Scanner scanner) {
        int id = scanner.nextInt();
        scanner.nextLine();
        String name = scanner.nextLine();
        String department = scanner.nextLine();
        double salary = scanner.nextDouble();

        if (salary < 30000) {
```

```

        System.out.println("Salary must be at least 30000.");
        return;
    }

    String sql = "INSERT INTO employees (employee_id, name, department,
salary) VALUES (?, ?, ?, ?)";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, id);
        ps.setString(2, name);
        ps.setString(3, department);
        ps.setDouble(4, salary);
        ps.executeUpdate();
        System.out.println("Employee added successfully");
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}

public static void updateSalary(Connection conn, Scanner scanner) {
    int id = scanner.nextInt();
    double newSalary = scanner.nextDouble();

    if (newSalary < 30000) {
        System.out.println("Salary must be at least 30000.");
        return;
    }

    String sqlCheck = "SELECT * FROM employees WHERE employee_id = ?";
    String sqlUpdate = "UPDATE employees SET salary = ? WHERE employee_id
= ?";

    try (PreparedStatement psCheck = conn.prepareStatement(sqlCheck)) {
        psCheck.setInt(1, id);
        ResultSet rs = psCheck.executeQuery();

        if (!rs.next()) {
            System.out.println("Employee not found.");
            return;
        }
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}

```

```

        return;
    }

    try (PreparedStatement ps = conn.prepareStatement(sqlUpdate)) {
        ps.setDouble(1, newSalary);
        ps.setInt(2, id);
        ps.executeUpdate();
        System.out.println("Salary updated successfully");
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}

```

```

public static void viewEmployeeRecord(Connection conn, Scanner scanner) {
    int id = scanner.nextInt();
    String sql = "SELECT * FROM employees WHERE employee_id = ?";

    try (PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, id);
        ResultSet rs = ps.executeQuery();

        if (rs.next()) {
            System.out.println("ID: " + rs.getInt("employee_id") +
                " | Name: " + rs.getString("name") +
                " | Department: " + rs.getString("department") +
                " | Salary: " + String.format("%.2f", rs.getDouble("salary")));
        } else {
            System.out.println("Employee not found.");
        }
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}

```

```

public static void displayAllEmployees(Connection conn) {
    String sql = "SELECT * FROM employees";
    try (Statement st = conn.createStatement()) {
        ResultSet rs = st.executeQuery(sql);
        System.out.println("ID | Name | Department | Salary");
        while (rs.next()) {
            System.out.println(rs.getInt("employee_id") + " | " +
                rs.getString("name") + " | " +

```

```

        rs.getString("department") + " | " +
        String.format("%.2f", rs.getDouble("salary"))));
    }
} catch (SQLException e) {
    System.out.println("Database Error: " + e.getMessage());
}
}

public static void main(String[] args) {
    String url = "jdbc:mysql://localhost/ri_db";
    String username = "test";
    String password = "test123";

    try (Connection conn = DriverManager.getConnection(url, username,
password);
        Scanner scanner = new Scanner(System.in)) {

        int choice;
        do {
            choice = scanner.nextInt();

            switch (choice) {
                case 1 -> addEmployee(conn, scanner);
                case 2 -> updateSalary(conn, scanner);
                case 3 -> viewEmployeeRecord(conn, scanner);
                case 4 -> displayAllEmployees(conn);
                case 5 -> System.out.println("Exiting Employee Management
System.");
                default -> System.out.println("Invalid choice. Please try again.");
            }

        } while (choice != 5);

    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Create a JDBC-based Inventory Management System that handles runtime input to manage items in an inventory. The system should allow users to:

Add a new item (item ID, name, quantity, price).

Restock an item by increasing its quantity.

Reduce the stock of an item, ensuring sufficient quantity.

Display all items in the inventory in a sorted order by item ID.

Exit the application.

Half of the code is given here; Only the remaining part should be completed.

The system should connect to a MySQL database using the following default credentials:

DB URL: jdbc:mysql://localhost/ri_db

USER: test

PWD: test123

The items table has already been created with the following structure:

Table Name: items

Input Format

The first line of input consists of an integer choice, representing the operation to be performed (1 for Add Item, 2 for Restock item, 3 for reduce item, 4 for Display, 5 for Exit).

For choice 1 (Add Item):

- The second line consists of an integer item_id.

- The third line consists of a string name.
- The fourth line consists of an integer quantity.
- The fifth line consists of a double price.

For choice 2 (Restock Item):

- The second line consists of an integer item_id.
- The third line consists of an integer quantity_to_add (must be positive).

For choice 3 (Reduce Stock):

- The second line consists of an integer item_id.
- The third line consists of an integer quantity_to_remove (must be positive).

For choice 4 (Display Inventory):

- No additional inputs are required.

For choice 5 (Exit):

- No additional inputs are required.

Output Format

For choice 1 (Add Item):

- Print "Item added successfully" if the item was added.
- Print "Failed to add item." if the insertion failed.

For choice 2 (Restock Item):

- Print "Item restocked successfully" if the restock was successful.
- Print "Item not found." if the specified item ID does not exist.

For choice 3 (Reduce Stock):

- Print "Stock reduced successfully" if the stock reduction was successful.
- Print "Not enough stock to remove." if there is insufficient quantity.
- Print "Item not found." if the specified item ID does not exist.

For choice 4 (Display Inventory):

- Display each item on a new line in the format:

- ID | Name | Quantity | Price
- If no items are available, print nothing (or handle with an appropriate message if desired).

For choice 5 (Exit):

- Print "Exiting Inventory Management System."

For invalid input:

- Print "Invalid choice. Please try again."

Sample Test Case

Input: 1

101

Laptop

50

1200.00

4

5

Output: Item added successfully

ID | Name | Quantity | Price

101 | Laptop | 50 | 1200.00

Exiting Inventory Management System.

Answer

```
import java.sql.*;
```

```
import java.util.Scanner;
```

```
class InventoryManagementSystem {
```

```
    public static void main(String[] args) {
```

```
        try (Connection conn = DriverManager.getConnection("jdbc:mysql://localhost/ri_db", "test", "test123");
```

```
            Scanner scanner = new Scanner(System.in)) {
```

```
            boolean running = true;
```

```
            while (running) {
```

```
                int choice = scanner.nextInt();
```

```
                switch (choice) {
```

```

        case 1:
            addItem(conn, scanner);
            break;
        case 2:
            restockItem(conn, scanner);
            break;
        case 3:
            reduceStock(conn, scanner);
            break;
        case 4:
            displayInventory(conn);
            break;
        case 5:
            System.out.println("Exiting Inventory Management System.");
            running = false;
            break;
        default:
            System.out.println("Invalid choice. Please try again.");
    }
}
} catch (SQLException e) {
    e.printStackTrace();
}
}

public static void addItem(Connection conn, Scanner scanner) {
    int id = scanner.nextInt();
    scanner.nextLine();
    String name = scanner.nextLine();
    int quantity = scanner.nextInt();
    double price = scanner.nextDouble();

    try {
        String sql = "INSERT INTO items (item_id, name, quantity, price) VALUES
        (?, ?, ?, ?)";
        PreparedStatement stmt = conn.prepareStatement(sql);
        stmt.setInt(1, id);
        stmt.setString(2, name);
        stmt.setInt(3, quantity);
        stmt.setDouble(4, price);

        int rows = stmt.executeUpdate();
        if (rows > 0) {

```



```

        System.out.println("Item added successfully");
    } else {
        System.out.println("Failed to add item.");
    }
} catch (SQLException e) {
    System.out.println("Failed to add item.");
}
}

```

```

public static void restockItem(Connection conn, Scanner scanner) {
    int id = scanner.nextInt();
    int quantityToAdd = scanner.nextInt();

```

```

    try {
        String check = "SELECT quantity FROM items WHERE item_id = ?";
        PreparedStatement checkStmt = conn.prepareStatement(check);
        checkStmt.setInt(1, id);
        ResultSet rs = checkStmt.executeQuery();

        if (!rs.next()) {
            System.out.println("Item not found.");
            return;
        }

        int newQuantity = rs.getInt(1) + quantityToAdd;
        String update = "UPDATE items SET quantity = ? WHERE item_id = ?";
        PreparedStatement updateStmt = conn.prepareStatement(update);
        updateStmt.setInt(1, newQuantity);
        updateStmt.setInt(2, id);
        updateStmt.executeUpdate();

        System.out.println("Item restocked successfully");

    } catch (SQLException e) {
        System.out.println("Item not found.");
    }
}

```

```

public static void reduceStock(Connection conn, Scanner scanner) {
    int id = scanner.nextInt();
    int quantityToRemove = scanner.nextInt();

```

```

try {
    String check = "SELECT quantity FROM items WHERE item_id = ?";
    PreparedStatement checkStmt = conn.prepareStatement(check);
    checkStmt.setInt(1, id);
    ResultSet rs = checkStmt.executeQuery();

    if (!rs.next()) {
        System.out.println("Item not found.");
        return;
    }

    int currentQuantity = rs.getInt(1);

    if (currentQuantity < quantityToRemove) {
        System.out.println("Not enough stock to remove.");
        return;
    }

    int newQuantity = currentQuantity - quantityToRemove;
    String update = "UPDATE items SET quantity = ? WHERE item_id = ?";
    PreparedStatement updateStmt = conn.prepareStatement(update);
    updateStmt.setInt(1, newQuantity);
    updateStmt.setInt(2, id);
    updateStmt.executeUpdate();

    System.out.println("Stock reduced successfully");
} catch (SQLException e) {
    System.out.println("Item not found.");
}

}

public static void displayInventory(Connection conn) {
    try {
        String sql = "SELECT * FROM items ORDER BY item_id";
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery();

        System.out.println("ID | Name | Quantity | Price");
        while (rs.next()) {
            System.out.println(rs.getInt("item_id") + " | " +
                rs.getString("name") + " | " +

```

```
        rs.getInt("quantity") + " | " +  
        String.format("%.2f", rs.getDouble("price"))));  
    }  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 12_Q1

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Sabrina is working on a project that involves analyzing a set of numbers. In her exploration, she encounters scenarios where extracting even numbers and finding their sum is essential.

Create a program that calculates the sum of even numbers from a given array of integers using a lambda expression.

Input Format

The first line of input consists of an integer N, representing the size of the array.

The second line consists of N space-separated integers, representing the elements of the array.

Output Format

The output prints the sum of the even integers from the array.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

29 37 45

Output: 0

Answer

// You are using Java

import java.util.*;

import java.util.stream.*;

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
  
        int n = sc.nextInt();  
        int[] arr = new int[n];
```

```
  
        for (int i = 0; i < n; i++) {  
            arr[i] = sc.nextInt();  
        }
```

```
  
        int evenSum = Arrays.stream(arr)  
            .filter(x -> x % 2 == 0)  
            .sum();
```

```
  
        System.out.println(evenSum);
```

```
        sc.close();
```

```
    }  
}
```

Status : Correct

Marks : 10/10

Rajalakshmi Engineering College

Name: Mohamed Aadhil
Email: 241801160@rajalakshmi.edu.in
Roll no: 241801160
Phone: 8056075484
Branch: REC
Department: AI & DS - Section 4
Batch: 2028
Degree: B.E - AI & DS

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 12_Q2

Attempt : 1
Total Mark : 10
Marks Obtained : 10

Section 1 : Coding

1. Problem Statement

Alex is learning about Java's functional interfaces and lambda expressions.

He wants to write a simple program that prints the square of each number in an array using a predefined functional interface.

Help Alex complete this task using the Consumer functional interface.

Input Format

- The first line contains an integer N, the number of elements in the array.
- The second line contains N space-separated integers.

Output Format

- Print the squares of all elements in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 2 3 4

Output: 1 4 9 16

Answer

// You are using Java

import java.util.*;

import java.util.function.Consumer;

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

```
  
        int n = sc.nextInt();  
        int[] arr = new int[n];
```

```
  
        for (int i = 0; i < n; i++) {  
            arr[i] = sc.nextInt();  
        }
```

```
  
        Consumer<Integer> printSquare = x -> System.out.print((x * x) + " ");
```

```
  
        for (int num : arr) {  
            printSquare.accept(num);  
        }
```

```
  
        sc.close();  
    }  
}
```

Status : Correct

Marks : 10/10